

README

Part 1：从零构建 Bigram 字符级语言模型

> 本节对应 Andrej Karpathy 的 [makemore 系列第 1 讲](https://www.youtube.com/watch?v=PaCmpygFfXo)，用中文重新讲解，配合可运行的代码脚本。

章节导航

序号	主题	内容概要
01	[前置知识与课程预告](01_introduction.md)	你需要知道什么、学完能做什么、语言模型是什么
02	[Bigram 模型：从统计到采样](02_bigram_model.md)	Bigram 概念、频率统计、概率矩阵、采样生成、NLL Loss
03	[用神经网络重新实现](03_neural_network.md)	one-hot 编码、Softmax、梯度下降、与计数法的等价性

配套资源

- 代码脚本：[\[../scripts/\]\(../scripts/\)](#) — 每个关键步骤都有独立 Python 脚本
- 输出图片：[\[../images/\]\(../images/\)](#) — Jupyter Notebook 的输出截图
- 课后作业：[\[../assignments/assignment_1/\]\(../assignments/assignment_1/\)](#) — 动手实践

学习路线

01 引言 ——→ 02 Bigram 统计模型 ——→ 03 神经网络版
(理解背景) (核心：计数与采样) (进阶：可扩展框架)

|



课后作业
(动手巩固)

建议按顺序阅读，每节大约 15-30 分钟。遇到代码片段时，打开对应的脚本文件一起看效果更好

[下一章：Part 2 MLP →](../Part2_mlp/tutorial/README.md)

01_introduction

01 前置知识与课程预告

前置知识

在开始之前，你需要对以下内容有基本了解：

Python 基础

会用 Python 写简单的循环、列表、字典就行。比如：

```
`python
```

列表与字典

```
words = ["emma", "olivia", "ava"]  
char_count = {"e": 1, "m": 2, "a": 1}
```

循环遍历

```
for word in words:  
    print(word)
```

如果你之前没写过 Python，推荐花 1-2 小时过一遍 [Python 官方教程](https://docs.python.org/zh-cn/3/tutorial/) 的前 4 章。

PyTorch 基础

我们会用 PyTorch 的 tensor（张量）来做计算。Tensor 就像 NumPy 的 ndarray，但可以在 GPU 上跑、支持自动求导。

核心概念就这几个：

```
`python  
import torch
```

创建 tensor

```
a = torch.zeros(3, 3) # 3×3 全零矩阵  
b = torch.tensor([1., 2.]) # 一维 tensor
```

基本运算

```
c = a + b # 逐元素加法  
d = a @ b # 矩阵乘法
```

形状操作

```
e = a.sum(dim=1, keepdims=True) # 按行求和，保持维度
```

安装 PyTorch：

```
`bash  
pip install torch
```

> 不需要精通 PyTorch，本教程会逐步解释用到的每一个操作。

概率论基础

几个关键概念：

概念	通俗解释
概率分布	每个事件发生的可能性，加起来等于 1
对数 (log)	把乘法变成加法的工具， $\log(a \times b) = \log(a) + \log(b)$
似然 (likelihood)	给定模型，观察到数据的概率
自然对数	底数为 e 的对数，PyTorch 里 <code>torch.log()</code> 就是自然对数

> 重点记住：概率相乘 $\rightarrow$ 取 log $\rightarrow$ 相加 $\rightarrow$ 取负，这就是后面 NLL Loss 的推导链条。

课程预告：学完这节课你能做什么？

学完 Part 1，你将：

1. 理解语言模型的基本框架 — 输入上下文，预测下一个 token
2. 用频率统计构建 Bigram 模型 — 从数据中统计字符对出现次数
3. 用神经网络重新实现并训练 — 同样的任务，不同的方法
4. 理解 Softmax 和 NLL Loss — 深度学习中最基础的两个概念
5. 自己生成名字 — 模型能"创作"出看起来像英文名的新名字

什么是语言模型？

语言模型的任务很简单：给定已经看到的内容，预测接下来会出现什么。

这就像你用手机打字时，键盘上方弹出的候选词：

你输入："今天天气"

键盘预测：["真好", "不错", "太热了", ...]

你输入："I love "

键盘预测：["you", "it", "coding", ...]

语言模型就是干这件事的。它学了"什么样的组合是常见的"，然后用来预测和生成文本。

在 Part 1 中，我们做一个最简单的语言模型 — 每次只看前一个字符，预测下一个字符。这就是 Bigram 模型。

数据集介绍

我们用的数据集是 `names.txt`，里面是一堆英文名字：

```
,
emma
olivia
ava
isabella
sophia
...
,
```

> 完整的探索脚本见 `../scripts/01_explore_data.py`

数据集规模大约 32,000 个名字，包含约 228,000 个字符。字符集是 26 个小写字母加上特殊的开始/结束标记

```
..o
```

我们选名字作为数据集是因为：

- 足够小 — 训练和实验速度快
- 足够有趣 — 生成的名字看起来"像真的"
- 结构清晰 — 每个名字是独立的、短小的序列

准备好了吗？ 下一节我们正式开始构建 Bigram 模型：[02 Bigram 模型：从统计到采样](`02_bigram_model.md`)

02_bigram_model

02 Bigram 模型：从统计到采样

这是 Part 1 的核心部分。我们从最朴素的方法开始 —— 统计字符对的出现频率，然后从频率中采样生成新名字。

1 Bigram 是什么

Bigram 就是"相邻的两个字符组成的一对"。对于一个名字，我们在开头和结尾各加一个特殊字符 `..`，然后拆成 bigram：

```
,
```

单词 "emma" 中的 bigram:

```
.e (开始→e)
em (e→m)
mm (m→m)
ma (m→a)
a. (a→结束)
```

```
,
```

Bigram 模型的假设非常简单粗暴：

> 下一个字符是什么，只取决于当前这一个字符。

这就是一个"马尔可夫链"——没有记忆，只看现在。虽然简单，但它是理解语言模型的最佳起点。

2 统计 Bigram 频率

我们的字符集有 26 个字母 + 1 个特殊字符 `.` = 27 个字符。

对数据集中所有名字统计 bigram 出现次数，可以构建一个 27×27 的计数矩阵 N ：

- $N[i][j]$ = 字符 i 后面跟着字符 j 的次数
- 行表示"当前字符"，列表示"下一个字符"

```
`python
```

核心逻辑（简化版）

```
N = torch.zeros(27, 27, dtype=torch.int32)
```

```
for word in words:
```

```
    chars = ['.'] + list(word) + ['.']
```

```
    for ch1, ch2 in zip(chars, chars[1:]):
```

```
        ix1 = stoi[ch1] # 字符 → 索引
```

```
        ix2 = stoi[ch2]
```

```
        N[ix1, ix2] += 1
```

```
`
```

> 完整脚本见 [[../scripts/02_bigram_counting.py](#)](../scripts/02_bigram_counting.py)

统计完之后，可视化一下这个矩阵：

```
`python
```

可视化：Bigram 计数矩阵热力图 → 生成 ../images/cell011_output00.png

```
import matplotlib.pyplot as plt
```

```
plt.figure(figsize=(16, 16))
```

```
plt.imshow(N, cmap='Blues')
```

```
for i in range(27):
```

```
    for j in range(27):
```

```
        chstr = itos[i] + itos[j]
```

```
        plt.text(j, i, chstr, ha="center", va="bottom", color='gray')
```

```
        plt.text(j, i, N[i, j].item(), ha="center", va="top", color='gray')
```

```
plt.axis('off')
```

```
plt.savefig('../images/cell011_output00.png', dpi=150, bbox_inches='tight')
```

```
plt.show()
```

```
`
```

> 完整脚本见 [[scripts/03_visualize_matrix.py](#)](../scripts/03_visualize_matrix.py)

![Bigram 计数矩阵热力图](../images/cell011_output00.png)

> 亮点解读：第一行（以 `.` 开头的行）告诉你哪些字母最常作为名字的开头。你能看到 **a**、**e**、**k** 等字母特别亮，说明很多名字以它们开头。

3 从计数到概率

有了计数矩阵 N ，转换成概率很简单：每一行归一化。

```
`python
P = N.float()
P /= P.sum(1, keepdims=True)
```

这一行代码做了什么？让我们拆开看：

Broadcasting 速成

`P /= P.sum(1, keepdims=True)` 这一行用到了 PyTorch 的 Broadcasting（广播）机制。这是后续所有 tensor 操作的基础，我们花点时间讲清楚。

什么是 Broadcasting？

当两个 tensor 形状不同时，PyTorch 会自动“广播”较小的 tensor，使其形状匹配较大的 tensor，然后逐元素运算。

广播规则（从右向左对齐）：

1. 从最右边的维度开始对齐
2. 每个维度必须满足以下条件之一：
 - 两个维度相等
 - 其中一个维度为 1
 - 其中一个维度不存在（会被补成 1）

我们的例子：

```
`
P 的形状：(27, 27)
P.sum(1, keepdims=True) 的形状：(27, 1)
```

广播后：(27, 27) $\leftarrow$ 第 2 维从 1 扩展到 27

具体发生了什么：

```
`
P = [[1, 2, 3], P.sum(1, keepdims=True)] = [[6],
[4, 5, 6]] [15]]
```

广播后，[6] 被复制成 [6, 6, 6]，[15] 被复制成 [15, 15, 15]：

```
`
P / P.sum = [[1/6, 2/6, 3/6],  $\leftarrow$  每个元素除以它所在行的总和
[4/15, 5/15, 6/15]]  $\leftarrow$  每个元素除以它所在行的总和
```

为什么需要 `keepdims=True`？

```
`python
```

没有 `keepdims`：形状从 (27,27) 变成 (27,) — 丢失了维度信息

`P.sum(1)  $\rightarrow$  shape (27,)` $\leftarrow$ 无法广播！(27,27) / (27,) 会报错

有 `keepdims`：形状从 (27,27) 变成 (27,1) — 保留了维度

`P.sum(1, keepdims=True) → shape (27, 1) ← 可以广播！(27,27) / (27,1)`

> 记忆技巧：`keepdims=True` 保持"形状的骨架"不变，只是把该维度的大小变成 1。这样后续做除法时，PyTorch 知道该往哪个方向广播。

这样 `P[i][j]` 就变成了：已知当前字符是 `i`，下一个字符是 `j` 的概率。

⚠ 注意 `keepdims=True` 很重要！如果省略，`sum` 会返回形状 `(27,)`，PyTorch 会按列广播，结果就全错了。这是个经典 bug。

> 完整的概率计算和采样脚本见 `[../scripts/04_probability_sampling.py](../scripts/04_probability_sampling.py)`

4 采样生成名字

有了概率矩阵 `P`，我们可以用它来生成新名字：

```
`python
g = torch.Generator().manual_seed(2147483647)

for i in range(5):
    out = []
    ix = 0 # 从特殊字符 '!' 开始
    while True:
        p = P[ix] # 取出当前字符对应的概率分布
        ix = torch.multinomial(p, num_samples=1, replacement=True, generator=g).item()
        if ix == 0: # 采样到 '!' → 结束
            break
        out.append(itos[ix]) # 索引 → 字符
    print(''.join(out))`
```

生成过程：

```
`
起始 → 查 P[0] ( . 的行) → 采样 → 得到 'j'
→ 查 P[10] (j 的行) → 采样 → 得到 'u'
→ 查 P[21] (u 的行) → 采样 → 得到 'n'
→ ... 直到采样到 '!' → 输出 "jun"`
```

> `torch.multinomial` 就是"按照给定的概率分布，随机抽一个"——就像加权抽奖。

生成的名字大概长这样（示例输出，实际结果可能因 PyTorch 版本略有不同）：

```
`
junide
janasah
p
cony
a`
```

> 上面的示例来自 Karpathy 原始 notebook。你运行

`[04_probability_sampling.py](../scripts/04_probability_sampling.py)` 时，输出的名字可能略有不同（因为 PyTorch

不同版本的 `multinomial` 实现有细微差异），但整体质量是一样的。

能看出有些像名字（junide、cony），有些很奇怪（单字母 `p`）。这就是 Bigram 模型的水平——它只能看到前一个字符，信息量太少了。后续课程会逐步改进。

5 评估模型质量：NLL Loss

生成的名字看起来还行，但我们需要一个数字化的指标来衡量模型好坏。

从似然到 NLL

思路：模型应该给训练数据中实际出现的 bigram 赋予较高的概率。

对于一个名字 "emma"：

似然 = $P(e|.) \times P(m|e) \times P(m|m) \times P(a|m) \times P(.,a)$
= 所有 bigram 概率的乘积

$\log \text{似然} = \log P(e|.) + \log P(m|e) + \log P(m|m) + \log P(a|m) + \log P(.,a)$
= 概率的 $\log$ 之和（乘法变加法！）

$\text{NLL} = -\log \text{似然}$
= 负的 $\log$ 似然

平均 NLL = $\text{NLL} / \text{bigram 总数}$ ← 这就是我们的 loss

关键理解：

量	越大越好还是越小越好？
似然（概率乘积）	越大越好
$\log$ 似然	越大越好（最大为 0）
NLL（负 $\log$ 似然）	**越小越好** （最小为 0）

我们用 NLL 作为 loss，是因为：

- 概率的乘积会导致数值下溢（一堆小于 1 的数相乘趋近于 0）
- 取 $\log$ 把乘法变加法，数值稳定
- 取负让优化目标统一为“最小化”

python

计算 NLL 的核心代码

```
log_likelihood = 0.0
n = 0

for word in words:
    chars = ['.'] + list(word) + ['.']
    for ch1, ch2 in zip(chars, chars[1:]):
        ix1 = stoi[ch1]
        ix2 = stoi[ch2]
        prob = P[ix1, ix2]
```



```
log_likelihood += torch.log(prob)
n += 1

nll = -log_likelihood
print(f"平均 NLL = {nll / n:.4f}") # 约 2.45
```

> 完整的 NLL 计算脚本见 [[../scripts/05_nll_loss.py](#)](../scripts/05_nll_loss.py)

模型平滑

△ 如果某个 bigram 在训练集中从未出现，它的计数为 0，概率就是 0。log(0) = $-\infty$ ，NLL 就炸了。

解决方法很简单：给所有计数加 1。

```
python
P = (N + 1).float() # 模型平滑
P /= P.sum(1, keepdims=True)
```

加 1 之后，所有 bigram 的概率都 > 0 ，不会出现 log(0)。这叫 Laplace 平滑（也叫 add-one smoothing）。+1 的大小控制了平滑的力度——加得越多，分布越均匀；加得越少，越接近原始计数。

课后练习

在进入下一节之前，想想这两个问题：

Q1：如果不加模型平滑 (N+1)，对于训练集中从未出现的 bigram 会发生什么？

<details>

<summary> 提示</summary>

考虑当 $N[i][j] = 0$ 时， $P[i][j] = 0$ ，然后 $\log(0) = ?$ 。

</details>

Q2：为什么用 NLL 而不是直接用似然作为 loss？

<details>

<summary> 提示</summary>

想想两个原因：(1) 概率相乘的数值稳定性；(2) 优化方向的一致性（我们总说“最小化 loss”）。

</details>

下一节，我们用神经网络来做同样的事：[03 用神经网络重新实现 Bigram](03_neural_network.md)

03_neural_network

03 用神经网络重新实现 Bigram

上一节我们用直接计数的方法构建了 Bigram 模型。这一节，我们要用神经网络做完全一样的事。

你可能会问：为什么要用神经网络重新造轮子？

> 答案：可扩展性。

>

> 直接计数法在 Bigram（只看 1 个字符）时很好用，但如果想看 3 个、5 个甚至更多字符的历史呢？组合数爆炸式增长，计数矩阵会变得巨大且稀疏。而神经网络的框架天然支持更复杂的输入，后续课程中我们只需要做微小改动就能升级模型。

1 构建训练数据：One-Hot Encoding

神经网络的输入是数字，不能直接吃字符。我们需要把字符转换成数值向量。

One-hot 编码：用一个 27 维向量表示一个字符，只有对应位置是 1，其余都是 0。

```
`
输入字符 '.' (索引0) → one-hot [1,0,0,...,0] (27维)
输入字符 'e' (索引5) → one-hot [0,0,0,0,0,1,...,0] (27维)
输入字符 'm' (索引13) → one-hot [0,0,...,1,...,0] (27维)
`
```

```
`python
import torch.nn.functional as F
```

创建训练数据

```
xs, ys = [], []
for word in words:
    chars = ['.'] + list(word) + ['.']
    for ch1, ch2 in zip(chars, chars[1:]):
        xs.append(stoi[ch1])
        ys.append(stoi[ch2])

xs = torch.tensor(xs)
ys = torch.tensor(ys)
```

one-hot 编码

```
xenc = F.one_hot(xs, num_classes=27).float() # 形状: (N, 27)
`
```

为什么 `.float()`？PyTorch 的 `one_hot` 默认返回整数类型，但神经网络需要浮点数来做矩阵乘法。

> 完整脚本见 `../scripts/06_neural_network.py` (`../scripts/06_neural_network.py`)

```
`python
```

可视化：One-hot 编码矩阵 → 生成
`../images/cell032_output01.png`

```
import matplotlib.pyplot as plt

plt.imshow(xenc) # xenc: (N, 27) 的 one-hot 矩阵
plt.colorbar()
plt.title('One-hot Encoding')
```

```
plt.xlabel('Character Index')
plt.ylabel('Sample Index')
plt.savefig('../images/cell032_output01.png', dpi=150, bbox_inches='tight')
plt.show()
```

![One-hot 编码可视化](../images/cell032_output01.png)

> 上图中，每一行是一个输入字符的 one-hot 表示。只有一列是黄色（值为 1），其余都是紫色（值为 0）。

2 前向传播：Softmax

我们用一个简单的单层神经网络：

```
输入 xenc (N, 27)
↓ 矩阵乘法
xenc @ W → logits (N, 27)
↓ 逐元素 exp
exp(logits) → counts (N, 27)
↓ 行归一化
counts / counts.sum(1, keepdims=True) → probabilities (N, 27)
```

这其实就是 Softmax：

```
python
```

初始化权重（ 27×27 矩阵，随机初始化）

```
W = torch.randn(27, 27, requires_grad=True)
```

前向传播

```
logits = xenc @ W # (N, 27) — 每个输入对应 27 个输出分数
counts = logits.exp() # (N, 27) — 确保非负（类比计数矩阵）
probs = counts / counts.sum(1, keepdims=True) # (N, 27) — 归一化为概率
```

直觉理解：

概念	计数版	神经网络版
计数矩阵	N（直接统计）	<code>logits.exp()</code> （通过 W 计算得到）
概率矩阵	<code>N / N.sum()</code>	<code>counts / counts.sum()</code>
参数	无（直接计数）	W（ 27×27 权重矩阵）

W 就是我们要学习的参数。训练的过程就是调整 W，使得模型输出的概率分布尽可能接近真实数据。

3 梯度下降训练

现在我们需要一个 loss 函数来衡量模型好坏 —— 就是上一节学过的 NLL（负对数似然）：

```
`python
```

对于每个训练样本，取出模型预测的目标字符概率

```
loss = -probs[torch.arange(len(ys)), ys].log().mean()
```

这行代码做了什么？

- `probs[torch.arange(len(ys)), ys]` — 取出每个样本中，目标字符对应的预测概率
- `.log()` — 取对数
- `.mean()` — 对所有样本取平均
- `-` — 取负，变成 NLL

训练循环：

```
`python
```

```
for k in range(100):
```

前向传播

```
logits = xenc @ W
counts = logits.exp()
probs = counts / counts.sum(1, keepdims=True)
loss = -probs[torch.arange(len(ys)), ys].log().mean()
```

反向传播

```
W.grad = None # 清零梯度
loss.backward() # 计算梯度
```

更新参数

```
W.data += -50 * W.grad # 学习率 50（这个任务比较简单，可以用大学习率）
```

```
if k % 10 == 0:
    print(f"Step {k}: loss = {loss.item():.4f}")
```

> 为什么学习率可以用 50？这是一个特例，不是通用经验！原因：

- > 1. 任务简单：只有 $27 \times 27 = 729$ 个参数，loss 曲面很平滑
- > 2. 数据量大：22 万个样本，梯度估计很稳定
- > 3. 没有隐藏层：不存在梯度消失/爆炸问题

>

> 到了 Part 2（MLP），学习率就变成了 0.1，因为网络更深、更复杂。通用建议：从小学习率开始（如 0.001），观察 loss 变化，再逐步调大。

> 完整的梯度下降脚本见 [\[../scripts/07_gradient_descent.py\]\(../scripts/07_gradient_descent.py\)](#)

训练后 loss 应该收敛到约 2.47 左右 —— 和计数法的平均 NLL 几乎一样！

L2 正则化 $\approx$ 模型平滑

还记得上一节的模型平滑 $N + 1$ 吗？在神经网络版中，等价的做法是 L2 正则化：

```
`python
```

加上正则化项

```
reg_loss = 0.01 (W * 2).mean() # 用 .mean() 而非 .sum(), 使正则化强度与 NLL 量纲匹配  
loss = nll_loss + reg_loss
```

直觉：正则化惩罚 W 中过大的值，使得 $W.exp()$ （即“计数”）不会太极端 → 相当于让分布更平滑 → 等价于 $N + \lambda$ 。

正则化系数 **0.01** 就相当于平滑的力度：系数越大 → 分布越平滑 → 生成结果越“平庸”。

4 神经网络版 vs 计数版：为什么结果一样？

训练结束后，我们来看看 $W.exp()$ 长什么样：

```
`python
```

训练后的 W 取 $\exp$

```
learned_N = W.exp().detach()
```

和直接计数得到的 N 对比

它们几乎一模一样！

为什么？因为两者在优化同一个目标：

- 计数版：直接统计频率，隐式地最大化似然
- 神经网络版：通过梯度下降最小化 NLL，等价于最大化似然

两者都是对同样的数据做最大似然估计

(MLE)，只不过一个直接算，一个迭代优化。既然优化目标相同，最优解当然一样！

> 这也说明：对于 Bigram

模型，直接计数就是最优解。神经网络的优势不在这里，而在于它的框架可以轻松扩展到更复杂的模型。

课后练习

Q1：为什么神经网络版 Bigram 的最优解和直接计数一样？

<details>

<summary> 提示</summary>

两者都在做最大似然估计。计数法直接给出了 MLE

的闭式解；神经网络通过梯度下降逼近同一个解。目标相同，最优解自然相同。

</details>

Q2：如果把 W 初始化为全零，训练还会收敛吗？

<details>

<summary> 提示</summary>

会！全零初始化意味着一开始所有 bigram 概率相等（均匀分布）。梯度会打破对称性，W 会逐步更新。不过收敛速度可能比随机初始化慢一些。

</details>

计数模型 vs 神经网络对照表

维度	计数法（Part 1 前半）	神经网络法（Part 1 后半）
参数	计数矩阵 N (27×27)	权重矩阵 W (27×27)
归一化	<code>P = N / N.sum(1)</code>	<code>probs = softmax(logits)</code>
损失函数	<code>NLL = -log(P[ch1,ch2]).mean()</code>	<code>CrossEntropy = -probs[range, ys].log().mean()</code>
平滑	<code>N + λ</code> （Laplace 平滑）	L2 正则化 <code>0.01 * (W**2).mean()</code>
采样	<code>torch.multinomial(P[ix])</code>	<code>torch.multinomial(probs[ix])</code>
优势	简单、快速、有闭式解	可扩展、可加隐藏层、可处理更多上下文
劣势	只能看 1 个字符	需要调参、训练慢

> 关键洞察：两种方法在 Bigram 任务上数学等价——神经网络训练收敛后，W 的最优解和计数法的 $\log(P)$ 一致。但神经网络的优势在于可扩展性：Part 2 我们会加隐藏层，Part 3 加 BatchNorm，Part 5 变成 WaveNet——这些都是计数法做不到的。

课后作业

动手实践时间！去完成课后作业来巩固这节课的内容：

[课后作业 Assignment 1](../../assignments/assignment_1/)

下一课预告

Part 2 中，我们将引入 MLP（多层感知机），把上下文长度从 1 个字符扩展到 3 个字符。这意味着模型不再是 Bigram，而是一个能考虑更多历史信息的更强模型。

核心升级路线：

,

Part 1: Bigram (看 1 个字符) ← 你在这里

Part 2: MLP (看 3 个字符) ← 下一站

Part 3: ...更深的网络...

,

下节课见！